AI Team Orchestration

Client Deployment Playbook

Grok as the boss · Claude, Grok, Gemini and ChatGPT as workers
Custom worker roles for each client's business
Mode A: client's own subscriptions on the client's computer
Mode B (option): client's own API keys (BYOK) on a free cloud server

Item	Value
Document type	Step-by-step procedure, from packaging to handover and support
Prepared	7 November 2026
For	The consultant who installs and configures the system for each client
Users of the finished system	The client's own staff, on the client's own computers
Who pays the AI providers	The client, always: their subscriptions (Mode A) or their API keys (Mode B)

Verify. Product names, install commands, subscription plans, free-tier limits and terms of service change often. Items checked online around the preparation date are marked **(checked)**; everything else is from general knowledge and marked **verify** or **approx.** Re-check the vendor's own pages before each client install. This playbook describes a reference layout; map it onto your actual engine, which the author of this document has not seen.

Contents

1. What you are delivering

2. Ground rules: accounts, terms and data

3. How the system works

4. Phase 1 — Package your laptop system into a product

5. Phase 2 — Discovery and worker-role design with the client

6. Phase 3 — Pre-install checklist

7. Phase 4 — Install on the client's computer (Mode A)

8. Phase 5 — Testing and acceptance

9. Phase 6 — Handover and staff training

10. Phase 7 — Updates, support and backups

11. Mode B — BYOK on a free cloud server

12. Security and privacy checklist

13. Troubleshooting

14. Master checklists

Appendix A — Role card template

Appendix B — Client handover sheet

Appendix C — Questions to settle before your first paid client

1. What you are delivering

You have built an orchestration system on your laptop: a **boss** model (Grok) receives a request, breaks it into tasks, and distributes them to **worker** models (Claude, Grok, Gemini, ChatGPT) through their command-line tools (CLIs), with your own routing and load-balancing logic in between. This playbook turns that into a repeatable service: you install the same system on each client's computer, design worker roles that fit their business, and leave them with something their staff can use and you can maintain.

1.1 Two deployment modes

	Mode A — Desktop (primary)	Mode B — Cloud BYOK (option)
Runs on	The client's own computer(s)	A free cloud server the client owns
AI access	Client's own subscriptions, logged in to each CLI	Client's own API keys (pay per use)
Who uses it	Staff at that computer	Any staff member, from a browser, after login
Always on?	Only while the computer is on	Yes
Extra build work	None beyond packaging	A simple chat page in front of your engine
Best for	Small teams, one or a few power users	Teams who want shared access, or many seats

1.2 Vocabulary used in this playbook

Term	Meaning
Engine	Your routing / load-balancing code. Identical for every client.
Boss	Grok, acting as planner and dispatcher: decides which worker gets which task, then reviews and combines the results.
Worker	A model doing one kind of job, defined by a role card (see Appendix A).
Role card	A short file describing one worker: purpose, instructions, model, limits, and when a human must review.
Client profile	All role cards plus routing rules for one client.
Subscription	A monthly consumer or business plan (e.g. Claude Pro/Max, ChatGPT Plus, SuperGrok, Google AI plans) used by logging in to a CLI.
API key (BYOK)	A pay-per-use key from a provider's developer console.

2. Ground rules: accounts, terms and data

2.1 The client owns every AI account

- The client buys each subscription (or creates each API account) in **their** company's name, with **their** payment method and a company email address.
- The client types their own passwords and completes their own two-factor prompts during install. You never collect, store or reuse a client's passwords.
- If you stop working with the client, the system keeps working because nothing depends on your accounts.

2.2 Terms-of-service checks before you sell

Verify. I am not certain of each provider's current wording, so confirm these points on each provider's terms and plan pages, and keep a dated note of what you found:

- **Seats:** consumer subscriptions are generally licensed to one person. If several staff members will use the system, the client may need one seat each or a team/business plan. Sharing one login across several people may breach the terms.
- **CLI access is included:** confirm which plan tiers include CLI use (e.g. which Claude, ChatGPT, SuperGrok and Google plans unlock Claude Code, Codex, Grok Build and Gemini CLI respectively).
- **Automation:** confirm the plan allows its CLI to be driven by a script (your engine) rather than typed into by hand. Usage limits on subscriptions are sized for one person.
- **Business use:** confirm commercial use of outputs is allowed on the chosen plans.
- **Your own work:** you configured your engine with help from Grok; confirm xAI's terms on using generated output commercially before reselling setups built with it.

2.3 Client data

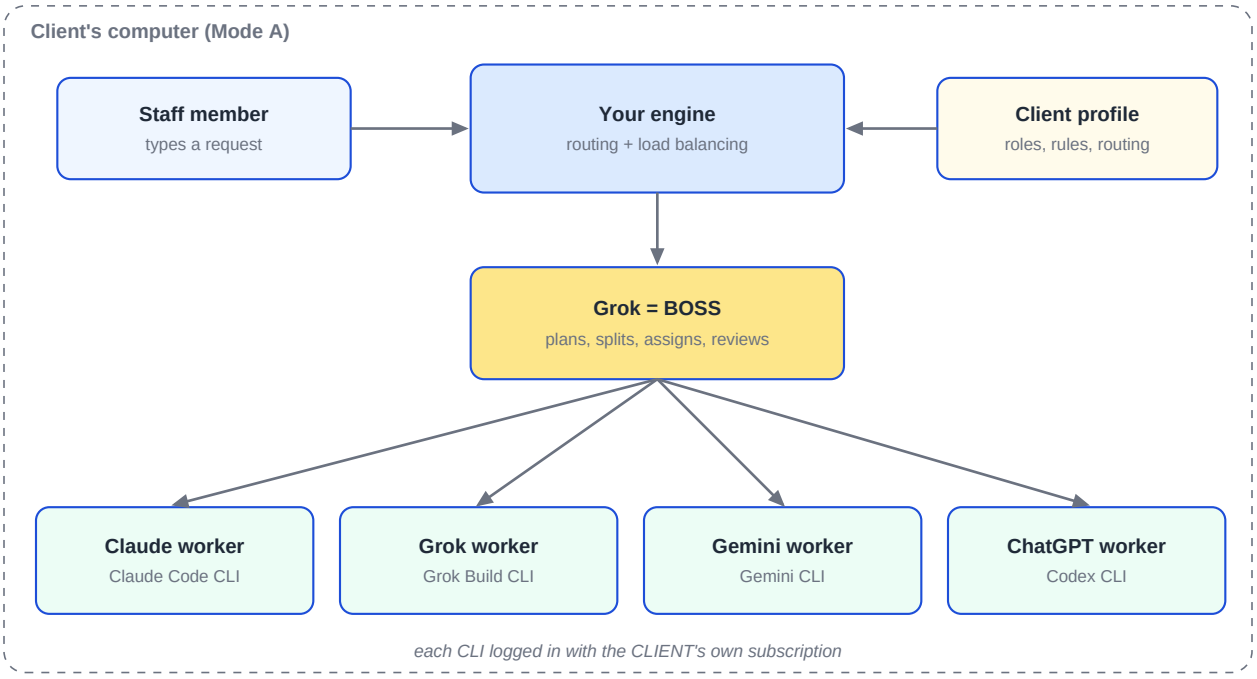
- Every request a worker handles is sent to that provider. List for the client which providers will see which kinds of data.
- Consumer plans may, depending on settings, allow the provider to use conversations to improve models; business plans and API usage often default differently. I do not have current verified details for each provider, so check each account's privacy/data settings with the client during install and record the choices.
- Agree in writing which data must never be sent to the system (e.g. card numbers, health records, passwords), and put that rule in every role card.

2.4 Your service agreement (outline)

Put a short written agreement in place; have a lawyer review it, as I cannot advise on your jurisdiction. Typical contents: scope (roles delivered, number of computers), the client's responsibility for their AI accounts and costs, data rules, support hours and response times, update policy, limits of liability for AI-generated output, and what happens at the end of the engagement.

3. How the system works

3.1 Mode A at a glance



3.2 Life of a request

1. A staff member gives the system a request (e.g. "Draft a quote for the Smith job and a follow-up email").
2. The engine sends it to **Grok (boss)** together with the list of available worker roles from the client profile.
3. The boss returns a plan: which tasks, which role handles each, in what order.
4. The engine dispatches each task to the right worker CLI, applying your load-balancing rules (e.g. if one provider hits its usage limit, use the role's fallback model).
5. Results come back to the boss, which checks them against each role's rules and combines them into one answer.
6. If a role is marked *human review required*, the result is shown as a draft for the staff member to approve rather than as final output.

3.3 Headless (non-interactive) CLI calls

For an engine to drive a CLI, each tool needs a non-interactive mode where a prompt goes in and an answer comes out. Your engine already does this; the table records the commands to confirm on each client install, because flags change between versions.

Worker	CLI	Non-interactive form (verify with --help)
Claude	Claude Code	`claude -p "<prompt>"`
ChatGPT	Codex CLI	`codex exec "<prompt>"`
Gemini	Gemini CLI	`gemini -p "<prompt>"`
Grok	Grok Build (`grok`)	Check `grok --help`; I have not verified its headless flag

Safety. These CLIs are built as coding agents and can read files and run commands in the folder they start in. Start each worker in a dedicated, empty working folder (e.g. `work/claude`) and use each tool's permission settings so workers cannot change files or run commands you did not intend.

4. Phase 1 — Package your laptop system into a product

Goal: anything client-specific or personal is pulled out of the code, so the same engine installs anywhere with a different profile.

4.1 Steps

1. **Back up** your current working system (copy the whole folder somewhere safe).
2. **Put it in a private GitHub repository** if it is not already. This becomes the master copy and the source of every client install and update.
3. **Find personal values**: your usernames, folder paths (e.g. `C:\Users\you\...` or `/Users/you/...`), your email, any keys or tokens. Replace each with a setting read from a config file or environment variable.
4. **Move roles and routing into files** under `profiles/` (format below), so a client's setup is data, not code.
5. **Add a .gitignore** so logs, outputs and any secrets file are never committed.
6. **Write the install script** (4.4) and test it on a clean computer or a fresh user account on your own machine. This is the single best predictor of a smooth client install.
7. **Tag a version** (e.g. v1.0.0) so you always know what each client is running.

4.2 Suggested folder layout

```
ai-team/
  engine/                # your routing + load balancing (same for all clients)
  profiles/
    _template/           # starter profile copied for each new client
      team.yaml          # boss, workers, routing rules
      roles/
        researcher.yaml
        writer.yaml
    acme-plumbing/       # one folder per client (or keep in a private client repo)
  scripts/
    install-mac.sh
    install-windows.ps1
    update.(sh|ps1)
    doctor.(sh|ps1)      # checks every CLI is installed and logged in
  docs/
    staff-guide.md       # one-page guide left with the client
  .gitignore             # logs/, outputs/, *.env
  VERSION
```

4.3 Team and role file formats (example)

Use whatever format your engine already reads; YAML is shown because it is easy for non-programmers to edit.

```
# profiles/acme-plumbing/team.yaml
client: Acme Plumbing Ltd
boss:
  worker: grok
  instructions: |
```

```
    You are the team lead. Split each request into tasks, assign each task to the
    best role below, then check and combine the results. Follow every role's rules.
workers:                    # how to reach each model on THIS machine
  claude:  { cli: claude, mode: subscription }
  chatgpt: { cli: codex,  mode: subscription }
  gemini:  { cli: gemini, mode: subscription }
  grok:    { cli: grok,   mode: subscription }
routing:
  max_parallel_tasks: 2       # keep low: subscription limits are per person
  retry_on_limit: true        # on a usage-limit error, use the role's fallback
  timeout_seconds: 300

# profiles/acme-plumbing/roles/quote-writer.yaml
id: quote-writer
title: Quote Writer
purpose: Turn job notes into a customer quote using the price list.
worker: claude
fallback: chatgpt
inputs: [job notes, price-list.csv]
output: Quote in the company template, under one page.
instructions: |
  Use only prices from price-list.csv. If an item is missing, list it under
  "Needs pricing" instead of guessing. Plain, friendly tone.
must_not:
  - invent prices, discounts or delivery dates
  - include customer card or bank details
human_review: required        # staff approve before anything is sent
```

4.4 Install script responsibilities

- Check prerequisites (Node.js version, Git) and stop with a clear message if missing.
- Install or update each CLI (Section 7.2).
- Copy the engine and the client's profile into a fixed location.
- Create the per-worker working folders.
- Run `doctor`: confirm each CLI starts and is logged in, and run one test prompt through the boss.
- Create a desktop shortcut or a single start command for staff.

5. Phase 2 — Discovery and worker-role design

This is where your value is: understanding the business and turning it into a small team of well-defined workers. Plan a 60–90 minute session with the owner and the staff who will use the system.

5.1 Discovery questions

- What tasks eat the most time each week? Who does them today?
- Which of those produce a document, message or decision that a person could check?
- What inputs exist (templates, price lists, FAQs, past examples)? Collect 3–5 good examples of finished work for each task.
- What must never happen (wrong prices, promises, sharing private data, tone mistakes)?
- Who approves output before it leaves the business?
- Which staff members will use the system, on which computers?

5.2 From tasks to roles

- Start with **3 to 5 roles**. More roles make the boss's routing less reliable and are harder to test.
- One role = one kind of output. Split a role if its instructions start to contradict themselves.
- Write the *must_not* list from the client's own words in discovery.
- Mark any customer-facing or money-related role as **human review required**.
- Assign a primary model and a fallback. Choose by testing the client's real examples on two or three models and comparing; model strengths shift with each release, so avoid fixed assumptions about which model is best at what.

5.3 Example role sets (starting points, adapt per client)

Business type	Possible roles
Trades / home services	Quote Writer, Job Scheduler Assistant, Customer Reply Drafter, Review Responder
Real estate	Listing Writer, Market Summary Researcher, Client Update Drafter, Document Checklist Checker
Accounting / bookkeeping	Client Email Drafter, Document Request Tracker, Policy Q&A (internal), Meeting Summariser
Retail / e-commerce	Product Description Writer, Customer Service Drafter, Social Post Writer, Competitor Price Researcher
Professional services	Proposal Writer, Research Assistant, Report Formatter, Meeting Summariser

Note. For anything legal, medical, financial or safety-related, set roles to draft-only with mandatory human review, and say so in the staff guide.

6. Phase 3 — Pre-install checklist

Send this to the client about a week before the install visit or screen-share.

6.1 Client tasks

[ ] Buy the agreed subscriptions in the company's name (one per user if required — see 2.2): Claude, ChatGPT, SuperGrok/X, Google AI plan as agreed.

[ ] Enable two-factor authentication on each account and have phones ready for codes.

[ ] Have the computer's administrator (admin) password available.

[ ] Collect the example documents, templates and price lists from discovery.

[ ] Decide which staff will use the system and on which computers.

6.2 Computer requirements

Item	Requirement
Operating system	Recent macOS, or Windows 10/11. On Windows, some CLIs work best inside WSL2 (Windows Subsystem for Linux) — check each tool's docs.
Node.js	A current LTS release. Sources disagree on minimum versions per CLI (one guide says Codex needs Node 22+); use the newest LTS to be safe **(verify)**.
Git	Needed to install and update your engine.
Internet	Stable connection; the system cannot work offline.
Disk / RAM	Several GB free disk; 8 GB RAM or more recommended (approx.).

7. Phase 4 — Install on the client's computer (Mode A)

7.1 Order of work

1. Create a dedicated local user folder for the system, or a separate computer account if the machine is shared.
2. Install prerequisites: Node.js LTS and Git.
3. Install the four CLIs (7.2).
4. **Client logs in to each CLI** with their own account while you guide (7.3).
5. Install your engine from your repository at the tagged version.
6. Copy in the client's profile (roles + routing) from Phase 2.
7. Run `doctor`, then the acceptance tests in Section 8.
8. Create the staff start shortcut; walk staff through one real task.

7.2 Install the CLIs

CLI	macOS / Linux	Windows (PowerShell)	Status
Claude Code	`curl -fsSL https://claude.ai/install.sh | bash`	`irm https://claude.ai/install.ps1 | iex`	verify; npm package `@anthropic-ai/claude-code` also exists
Grok Build	`curl -fsSL https://x.ai/cli/install.sh | bash`	`irm https://x.ai/cli/install.ps1 | iex`	checked (secondary sources); beta, changes often — see x.ai/cli
Gemini CLI	`npm install -g @google/gemini-cli`	same	checked (secondary sources)
Codex CLI	`npm install -g @openai/codex`	same	checked (secondary sources)

Verify. Piping an install script from the internet into a shell runs it with your permissions. Only use the official domains above, type them carefully, and prefer the vendor's current install page over copying from this document.

7.3 Log in with the client's subscriptions

CLI	How to sign in	What to check
Claude Code	Run `claude` and choose the Claude account (subscription) login	Signed in as the client's company email
Codex CLI	Run `codex login` and sign in with ChatGPT	Correct ChatGPT account and plan
Gemini CLI	Run `gemini` and choose Google account login	Company Google account, not a personal one
Grok Build	Run `grok` and sign in	Correct X/xAI account

- Make sure no API-key environment variables (e.g. `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `GEMINI_API_KEY`) are set on the machine in Mode A; some CLIs prefer a key over the subscription login when both exist, which would bill the client per use instead **(verify per CLI)**.
- Check each CLI's own logout/status command so you can show the client how to disconnect an account if a staff member leaves.

7.4 Smoke test

```
doctor                          # your script: versions + login status for all 4 CLIs
claude -p "Reply with OK"       # each worker answers on its own (verify flags)
codex exec "Reply with OK"
gemini -p "Reply with OK"
<your engine start command> "Plan a short test task using two roles"
```

8. Phase 5 — Testing and acceptance

8.1 Test set per role

For each role, prepare at least five test requests built from the client's real examples: three normal cases, one edge case (missing information), and one *must_not* case (a request that should be refused or flagged). Record the expected behaviour in advance.

Check	Pass when
Routing	The boss assigns each test request to the intended role
Quality	Output matches the client's example standard; staff would send it after light edits
Rules	Every must_not case is refused, flagged or left for a human
Fallback	Simulated limit/unavailable worker results in the fallback model being used, or a clear message
Speed	Typical request completes in a time staff accept (agree a target)
Review gates	Roles marked human_review produce drafts, never final sends

8.2 Acceptance

Run the tests with the client watching, fix what fails, re-run, and have the client sign the handover sheet (Appendix B). Keep the test set; rerun it after every update.

9. Phase 6 — Handover and staff training

- A one-page staff guide: how to start the system, how to phrase requests, what the roles are, what must never be entered, how to report a problem.

- A 30–45 minute session where each user completes two real tasks.
- Show the owner how to: see which accounts are logged in, sign a departing staff member out, and change a subscription plan.
- Leave the handover sheet (Appendix B) with versions, roles, contacts and support terms.

10. Phase 7 — Updates, support and backups

10.1 Updates

- Improve the engine in your master repository; tag a new version; test it on your laptop with a copy of each client's profile.
- Update a client with the update script (pull the tagged version, update CLIs, run doctor and the client's test set). Record the new version on the handover sheet.
- Update the CLIs themselves regularly (e.g. `grok update`, `npm update -g` for npm-installed tools); retest afterwards because CLI changes can break headless flags.

10.2 Remote support

- Use a remote-support tool the client controls, with access granted per session.
- Never ask for passwords; let the client re-authenticate CLIs themselves if needed.

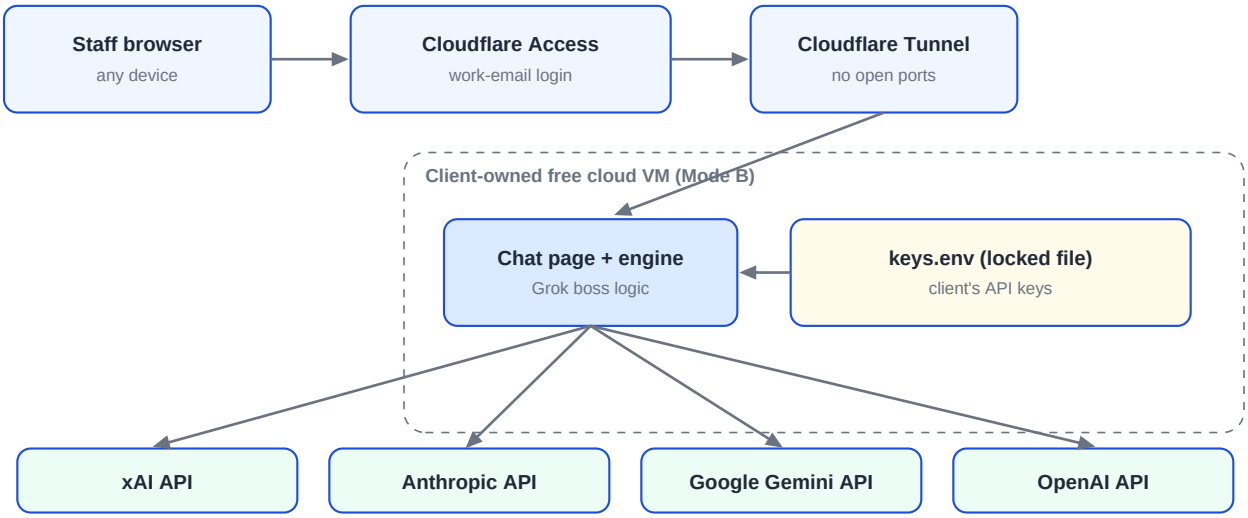
10.3 Backups

- Your master repository plus each client profile is the backup of the configuration.
- Agree whether outputs/logs are kept, where, and for how long; exclude them from Git.

11. Mode B — BYOK on a free cloud server

Offer Mode B when a client wants staff to use the system from any device, wants it running when office computers are off, or would need many subscription seats. The client creates **API accounts** (pay per use) instead of subscriptions, and the client owns the cloud server too, so the whole setup stays theirs.

11.1 Mode B at a glance



11.2 What changes compared to Mode A

- Workers use the client's **API keys** instead of subscription logins. Most of the CLIs can read a key from an environment variable (e.g. `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `GEMINI_API_KEY`); confirm Grok Build's API-key option in its docs. Alternatively your engine can call the providers' APIs directly, which is faster and more robust on a server.
- Staff reach the system through a simple **private chat web page**. This is the one new piece to build; your engine's start command sits behind it.
- Access is protected by a login (Cloudflare Access) and the server is not exposed directly to the internet (Cloudflare Tunnel).

11.3 Free server options

Option	What's free	Caveats
Oracle Cloud Always Free (recommended)	Arm VM (Ampere A1). Reported in 2026 as cut to **2 OCPU / 12 GB RAM** total, from the 4 OCPU / 24 GB many guides still quote **(checked, secondary sources — verify on Oracle's page)**	Sign-up needs a card for verification (verify). Idle instances can be reclaimed when usage stays low over 7 days (thresholds reported variously at ~10–20% CPU). Capacity can be unavailable in some regions.
Google Cloud free tier	One small e2-micro VM in certain US regions (approx.; verify)	About 1 GB RAM — tight for several Node-based CLIs plus an engine.
Render free web service	750 instance-hours/month **(checked)**	Sleeps after 15 min idle (~1 min wake-up) and is designed for web apps, not long-running CLI jobs — not recommended here.
Low-cost paid VPS	Not free: roughly $5–10/month (approx.)	Simplest reliable option if free capacity is unavailable.

11.4 Get the client's API keys

Provider	Where the client creates the key (verify URLs)	Environment variable
xAI (Grok)	console.x.ai	`XAI_API_KEY` (common convention; verify)
Anthropic (Claude)	console.anthropic.com	`ANTHROPIC_API_KEY`
Google (Gemini)	aistudio.google.com	`GEMINI_API_KEY`
OpenAI (ChatGPT models)	platform.openai.com	`OPENAI_API_KEY`

- The client creates each key while logged in to their own account and pastes it directly into the server's key file during the session (11.5 step 6). Do not send keys by email or chat.
- Have the client set a **monthly spending limit** or budget alert in each provider console, if the provider offers one.

11.5 Server setup (Oracle Always Free, Ubuntu)

1. Client creates the Oracle Cloud account (company email, their card for verification).
2. Create a VM: Ubuntu image, Ampere A1 shape within the Always Free allowance; add your SSH public key (and the client's) for login. Keep the default network rules; do not open web ports — the tunnel makes that unnecessary.
3. Log in over SSH and apply updates.
4. Create a dedicated user for the system and install Node.js LTS (follow nodejs.org's current Ubuntu instructions) and Git.
5. Install your engine and the API-key-capable CLIs (or your direct-API workers).
6. Create the locked key file with the client's keys.
7. Run the engine as a service so it restarts automatically.
8. Add Cloudflare Tunnel and Access (11.6).
9. Run the acceptance tests (Section 8) through the chat page.

```
# step 3-4
sudo apt update && sudo apt -y upgrade
sudo apt install -y git curl
sudo useradd --create-home --shell /bin/bash aiteam

# step 6: key file readable only by the service user
sudo mkdir -p /etc/ai-team
sudo nano /etc/ai-team/keys.env          # client pastes:  XAI_API_KEY=...  etc.
sudo chown aiteam:aiteam /etc/ai-team/keys.env
sudo chmod 600 /etc/ai-team/keys.env

# step 7: /etc/systemd/system/ai-team.service
[Unit]
Description=AI team engine
After=network-online.target
[Service]
User=aiteam
WorkingDirectory=/home/aiteam/ai-team
EnvironmentFile=/etc/ai-team/keys.env
ExecStart=/usr/bin/npm start              # replace with your engine's start command
Restart=on-failure
[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload && sudo systemctl enable --now ai-team
```

11.6 Private access: Cloudflare Tunnel + Access

Cloudflare's Zero Trust free plan covers up to **50 users** and includes Access and Tunnel **(checked)**. It needs a domain managed in a Cloudflare account (the client's). The tunnel connects outward from the server, so no inbound web port is opened.

```
# on the server (install cloudflared per Cloudflare's current docs first)
cloudflared tunnel login
cloudflared tunnel create ai-team
cloudflared tunnel route dns ai-team team.clientdomain.com
# config: send team.clientdomain.com to the chat page, e.g. http://localhost:3000
sudo cloudflared service install
```

- In the Cloudflare Zero Trust dashboard, create an Access application for `team.clientdomain.com` with a policy allowing only the client's staff email addresses (or their email domain).
- Staff then open the address, enter their work email, receive a one-time code, and reach the chat page.
- Command names above reflect Cloudflare's documented flow as I know it; follow the current Cloudflare Tunnel guide, which may prefer dashboard-created tunnels **(verify)**.

11.7 Keeping a free server healthy

- Check Oracle's idle-reclamation rules; a lightly used assistant may look idle. Upgrading the account to pay-as-you-go (still using free resources) is reported to avoid reclamation — verify before relying on it.
- Enable automatic security updates (e.g. Ubuntu's unattended-upgrades).
- Back up the engine folder and profile (they are already in Git) and the key file location (not the keys — the client can recreate keys).

12. Security and privacy checklist

[ ] All AI accounts are in the client's name; you hold no client passwords.

[ ] Two-factor authentication is on for every AI, cloud and Cloudflare account.

[ ] Mode A: no stray API-key environment variables; each worker runs in its own folder with restricted CLI permissions.

[ ] Mode B: keys only in the locked key file (chmod 600), never in Git, email or chat; spending limits set at each provider.

[ ] Mode B: no inbound web ports open; access only through Cloudflare Access.

[ ] Data rules agreed and written into every role card's must_not list.

[ ] Privacy/data-use settings reviewed in each AI account and recorded.

[ ] Customer-facing and money-related roles require human review.

[ ] Logs and outputs excluded from Git and kept only as long as agreed.

[ ] Process for removing a departing staff member (sign out CLIs / remove from Access policy).

13. Troubleshooting

Symptom	Likely cause	Fix
A worker says it is not logged in	Session expired or password changed	Client re-runs that CLI's login
Worker errors about usage limits	Subscription limit reached	Fallback should take over; consider a higher plan or Mode B for heavy use
Worker bills per use unexpectedly	An API-key variable is set in Mode A	Remove the variable; restart the engine
Boss sends tasks to the wrong role	Role purposes overlap or are vague	Sharpen role purposes; reduce the number of roles
Headless call hangs	CLI waiting for a permission or update prompt	Run the CLI by hand to see the prompt; adjust permission settings; update CLI
Everything broke after an update	CLI flags changed	Roll back to the previous tagged version; retest; fix the engine
Mode B page unreachable	Tunnel or service stopped; VM reclaimed	Check `systemctl status ai-team` and cloudflared; check the cloud console

14. Master checklists

14.1 New client

[ ] Service agreement signed (2.4)

[ ] Terms checks done and noted (2.2)

[ ] Discovery session held; examples collected (5.1)

[ ] Role cards written and reviewed with the client (5.2, Appendix A)

[ ] Pre-install checklist completed by client (6)

[ ] Install done; CLIs logged in by client (7)

[ ] Acceptance tests passed (8)

[ ] Staff trained; handover sheet signed (9, Appendix B)

14.2 Each update

[ ] New version tagged and tested on your laptop with the client's profile

[ ] Client notified of the update window

[ ] Update script run; doctor passes

[ ] Client test set passes

[ ] Version recorded on handover sheet

Appendix A — Role card template

```
id:            short-name-with-dashes
title:         Human-friendly name
purpose:       One sentence: what this worker produces, for whom.
worker:        claude | chatgpt | gemini | grok
fallback:      second choice model
inputs:        what it is given (files, notes, templates)
output:        format, length, tone, template to follow
instructions: |
  The detailed working instructions in the client's language and style.
examples:      paths to 3-5 good examples of finished work
must_not:
  - things it must never do (from discovery)
human_review:  required | optional
owner:         client staff member responsible for this role
```

Appendix B — Client handover sheet

Field	Value
Client / site	
Mode	A (desktop) / B (cloud BYOK)
Computers or server	
Engine version installed	
CLI versions (Claude Code / Codex / Gemini / Grok Build)	
AI accounts and plans (owner: client)	
Roles delivered	
Data rules agreed	
Privacy settings chosen per AI account	
Support contact and hours	
Acceptance tests passed on (date)	
Client signature / date	

Appendix C — Questions to settle before your first paid client

- Exactly which subscription plans give CLI access for each provider today, and whether scripted (headless) use is allowed on them (2.2)?
- One seat per staff member, or a team plan — what will each client need?
- Does your engine run on Windows as well as macOS, or will Windows clients need WSL2?
- How will you price: setup fee per client, per role, plus monthly support?
- Which roles become your standard library, reused across clients in the same industry?
- When does a client graduate from Mode A to Mode B?